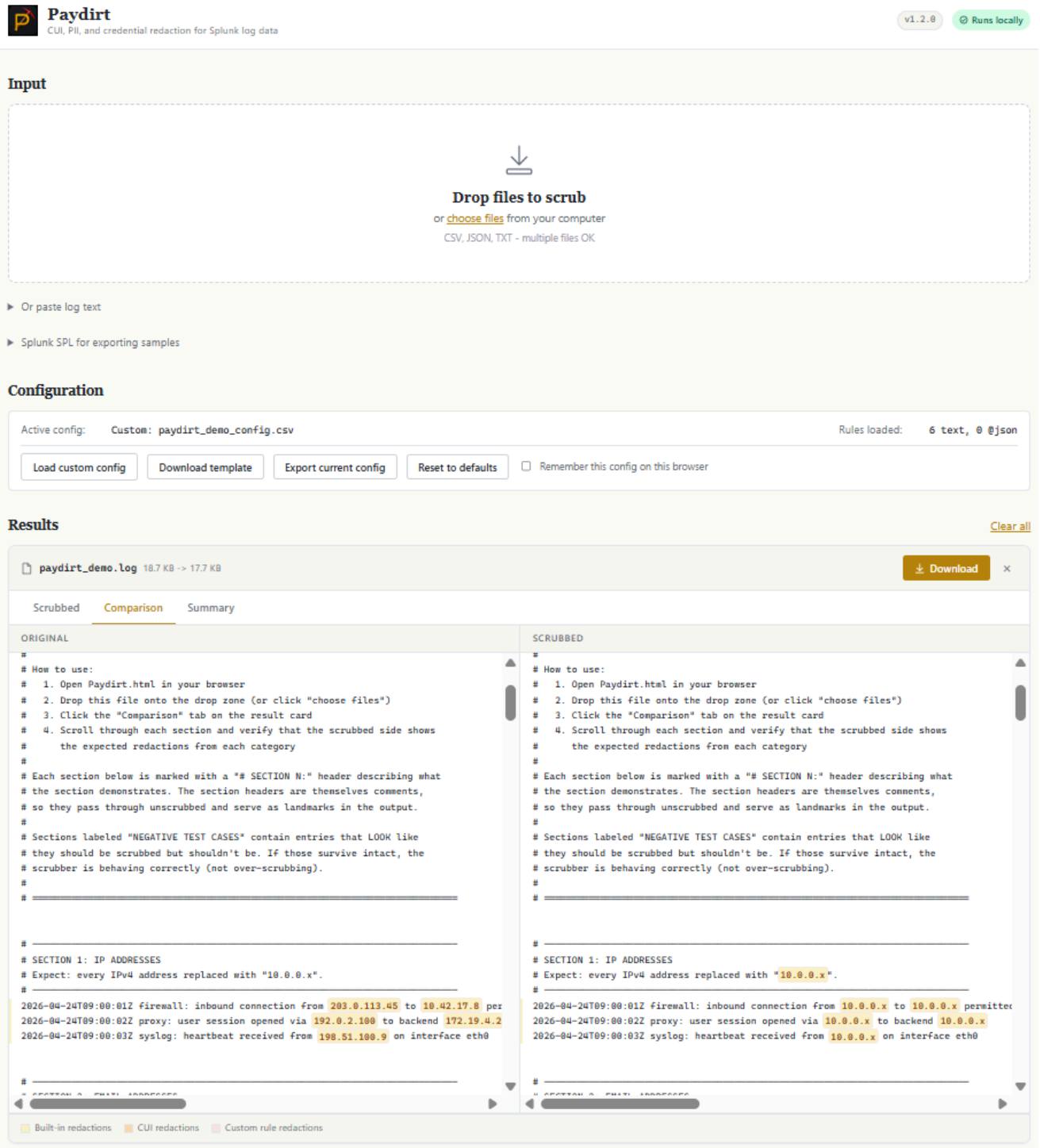
Paydirt + Log Scrubber

Paydirt is a self-contained browser tool plus matching Python CLI for scrubbing sensitive data from Splunk log exports so they can be mined for value. Drop a file, get a sanitized version - no install, no network calls, nothing leaves your machine. CMMC, HIPAA, and GDPR aware. By Machine Data Insights. *There's Gold in That Data!®*



Paydirt's Comparison view shows original and scrubbed output side by side, with per-field highlighting that tells you exactly what changed.

Download

Latest release: v1.2.0

Browser tool and config template:

- Paydirt.html - the browser tool. Save the file, double-click to open in your browser. No install required.
- log_scrubbing_config.csv - the default configuration file (optional - both tools have built-in defaults).

Demo files - click "Load custom config", load `paydirt_demo_config.csv`, then drop `paydirt_demo.log` onto the "Drop files to scrub" panel. Click "Comparison" to view all the redactions, then "Summary" to see an inventory of what was redacted by category.

- paydirt_demo.log - 25 sections covering every scrubbing category, with negative test cases that demonstrate what the scrubber correctly leaves alone.
- paydirt_demo_config.csv - companion config that targets Section 25 of the demo file (custom rules).

Python CLI for automation:

- log_scrubber.py - the Python CLI. Requires Python 3.9 or later. No other dependencies.

What It Does

Log scrubbing tools for removing CUI, PII, PHI, credentials, and other sensitive data from Splunk field-value exports and log samples. Two interchangeable implementations of the same scrubbing algorithm:

- **`Paydirt.html`** - a self-contained browser tool. Download the file, double-click, scrub logs in the browser. No install, no network calls, no data leaves your machine. Good for environments where you can't install Python (locked-down corporate laptops, air-gapped networks).
- **`log_scrubber.py`** - a Python CLI and importable library. Good for automation, batch processing, and integration into existing pipelines.

Both tools use the same configuration format (`log_scrubbing_config.csv`) and produce identical output for identical input (verified by `shared-tests/run_parity.py`).

Compliance coverage: CMMC (CUI marking detection per NIST SP 800-171), HIPAA (PHI identifiers including SSN, Luhn-validated payment cards, NPI, phone), GDPR (personal data including IPs and device identifiers).

No dependencies beyond Python 3.9+ standard library for the CLI, and no dependencies at all for the browser tool (pure HTML/CSS/JS, runs offline).

Documentation

- This README is the canonical project documentation
- `docs/LOG_SCRUBBER_GUIDE.md` - CLI quick-start, condensed for users who just want to scrub their first file
- `docs/COMPLIANCE_PRIMER.md` - plain-language compliance background

- `docs/SECURITY_ARCHITECTURE.pdf` - architecture and security review for compliance reviewers, covering data flow, trust boundaries, threat model, and code provenance
- `paydirt/README.md` - browser tool source layout and build process

Try It Yourself

The demo files (`paydirt_demo.log` and `paydirt_demo_config.csv`) are also included at the project root if you'd rather work from a cloned copy of the repo. To exercise the demo from the command line:

```
# Built-in patterns only:
python log_scrubber.py paydirt_demo.log

# Built-in patterns plus the companion config (lights up Section 25):
python log_scrubber.py paydirt_demo.log --config paydirt_demo_config.csv
```

Each section of the demo file is marked with a `# SECTION N:` header describing what it demonstrates. Reviewers evaluating the tool for compliance purposes can inspect each section's expected behavior, run the scrubber, and visually confirm that every category of sensitive data gets correctly redacted (and that negative test cases remain untouched).

Working With Your Own Splunk Data

To pull data from your own Splunk environment for scrubbing, Paydirt includes ready-to-run SPL examples directly in the UI. Click **Splunk SPL for exporting samples** in the Input section to expand a panel with two searches:

- **Field-value samples** (last 7 days) — uses `fieldsummary` to extract the distinct values present in each field across a sourcetype, with the housekeeping fields filtered out. This is the right input for understanding the shape of your data and building CIM normalization rules.
- **Log samples** (last 1 day) — uses `dedup punct | head 20` to grab a representative set of distinct event patterns. This is the right input for working with full event text, including any embedded JSON.

Each block has a **Copy** button to drop the SPL into your clipboard with one click. Substitute your own `{index}` and `{sourcetype}` values, run the search in Splunk Web, click **Export** → **CSV**, and drop the saved file onto Paydirt to scrub it.

The same searches work with `log_scrubber.py` — export from Splunk, then run `python log_scrubber.py your_export.csv`.

Repository Layout

```
paydirt/
├── README.md                   this file
├── Paydirt.html                the browser tool (single-file distribution)
├── log_scrubber.py             the Python CLI (single-file distribution)
├── log_scrubbing_config.csv    default config (shared by both tools)
│
├── paydirt_demo.log            demo log file for reviewers
```

```
├── paydirt_demo_config.csv      companion demo config
│
├── docs/                        compliance and architecture documentation
│   ├── COMPLIANCE_PRIMER.md     plain-language compliance background
│   ├── LOG_SCRUBBER_GUIDE.md    CLI quick-start guide
│   └── SECURITY_ARCHITECTURE.pdf architecture and security review document
│
├── cli/                         Python CLI tests
│   ├── README.md
│   └── test_scrubber.py
│
├── paydirt/                     browser tool source (build artifacts land
│   ├── README.md                at the repo root, not here)
│   ├── build.py
│   ├── src/
│   │   ├── index.html, styles.css, scrubber.js, app.js
│   └── tests/
│       ├── test-node.js         JS unit tests
│       └── smoke-test.py        headless browser smoke test
│
└── shared-tests/                parity tests between Python and JS
    ├── README.md
    ├── run_parity.py
    └── input/                   fixtures fed through both implementations
```

Quick Start - CLI

The remainder of this README covers the Python CLI. For the browser tool, see `paydirt/README.md` (for developers) or just open `Paydirt.html` in your browser (for users).

```
# Auto-detect mode from file contents (easiest)
python log_scrubber.py my_fields.csv

# Batch multiple files - mode auto-detected per file
python log_scrubber.py ~/Downloads/*.csv

# Explicit mode (v1 syntax, still supported)
python log_scrubber.py fieldsummary my_fields.csv
python log_scrubber.py samples my_events.csv

# Include raw values for comparison
python log_scrubber.py fieldsummary my_fields.csv --include-raw

# Disable CUI marking detection (if your data isn't CUI-regulated)
python log_scrubber.py my_events.csv --no-cui
```

Output is written to `my_fields_scrubbed_20260420_103045.csv` (auto-timestamped).

Output Format

Fieldsummary Mode

By default, the output contains **scrubbed values only** - the raw values column is removed so the output is safe to share:

```
Field Name, Scrubbed Values, Count, Distinct Count
```

Use `--include-raw` to keep the original raw values alongside the scrubbed values (useful for reviewing what was changed):

```
Field Name, Raw Values, Scrubbed Values, Count, Distinct Count
```

Samples Mode

Log sample events are scrubbed in place - the output file contains only scrubbed events.

Step-by-Step Workflow

1. Export Field Values from Splunk Web

Run this SPL in Splunk Web (adjust index, sourcetype, and time range):

```
index=<your_index> sourcetype="<your_sourcetype>" earliest=-7d@d latest=now
| fieldsummary maxvals=5
| search field!="_*" AND field!="date_*" AND field!="linecount"
  AND field!="punct" AND field!="timestartpos" AND field!="timeendpos"
  AND field!="splunk_server_group"
```

Click **Export** → choose **CSV** → save the file (e.g., `guardduty_fields.csv`).

2. Export Log Samples from Splunk Web

```
index=<your_index> sourcetype="<your_sourcetype>" earliest=-1d@d latest=now
| dedup punct | head 20
```

Click **Export** → choose **CSV** → save the file (e.g., `guardduty_samples.csv`).

Alternatively, you can copy/paste raw events into a plain `.txt` file (one event per line). The scrubber auto-detects the format.

3. Scrub the Exports

With auto-detection (recommended), you don't need to specify the mode:

```
# Scrub one file
python log_scrubber.py guardduty_fields.csv

# Scrub everything from today's Downloads
python log_scrubber.py ~/Downloads/*.csv
```

The scrubber inspects each file's header and dispatches to the correct mode. You can still use explicit mode keywords if you prefer:

```
python log_scrubber.py fieldsummary guardduty_fields.csv
python log_scrubber.py samples guardduty_samples.csv
```

4. Review and Send

Check the `*_scrubbed_*` output files to verify sensitive data was replaced, then email the scrubbed files for processing.

Configuration

Config File Location

The scrubber automatically looks for `log_scrubbing_config.csv` in these locations (first match wins):

1. Current working directory: `./log_scrubbing_config.csv`
2. Data subdirectory: `./data/log_scrubbing_config.csv`
3. Same directory as the script: `<script_dir>/log_scrubbing_config.csv`
4. Data subdirectory of script: `<script_dir>/data/log_scrubbing_config.csv`

The simplest approach is to **place the config file in the same folder as `log_scrubber.py`**.

You can also specify a config explicitly with `--config`:

```
python log_scrubber.py fieldsummary fields.csv --config /path/to/my_rules.csv
```

Built-in Scrubbing (Always Active)

Even without a config file, the scrubber applies these patterns automatically. They can be disabled as a group with `--no-builtin`.

Network & identity (v1.0):

Pattern	Replacement
IP addresses (`192.168.1.50`)	`10.0.0.x`
AWS ip- hostnames (`ip-10-50-26-117`)	`ip-10-0-0-x`

Pattern	Replacement
Email addresses (`admin@corp.com`)	`user@example.com`
FQDN hostnames (`server01.company.com`)	`host.example.com`
UNC paths (`\\server\share`)	`\\SERVER\SHARE`
Domain usernames (`CORP\jsmith`)	`DOMAIN\user`
MAC addresses (`00:1A:2B:3C:4D:5E`)	`00:00:00:00:00:00`

Credentials & tokens (v1.1):

Pattern	Replacement
PEM private key blocks	`[REDACTED_PRIVATE_KEY]`
AWS access keys (`AKIA...`, `ASIA...`)	`AKIAREDACTEDREDACTED`
GitHub PATs (`ghp_...`, `gho_...`, etc.)	`ghp_REDACTED`
Slack tokens (`xoxb-...`, `xoxp-...`, etc.)	`xoxb-REDACTED`
Stripe keys (`sk_live_...`, `sk_test_...`)	`sk_test_REDACTED`
JWTs (`eyJ...` 3-segment)	`eyJREDACTED.REDACTED.REDACTED`
Google API keys (`AIza...`)	`AIzaREDACTED`
HTTP `Authorization` header values	`Authorization: Bearer REDACTED`
URL query credentials (`?password=`, `&api_key=`, etc.)	`...=REDACTED`

PII / PHI (v1.1):

Pattern	Replacement
SSN (formatted `XXX-XX-XXXX`, valid ranges only)	`000-00-0000`
Credit card numbers (Luhn-validated)	`4111-1111-1111-1111`
NPI (10-digit, validated per 45 CFR 162.406)	`1234567893`
Phone numbers (formatted, US)	`555-555-5555`
Windows SIDs (user SIDs only, `S-1-5-21-...`)	`S-1-5-21-0-0-0-0`

Luhn and NPI validators run inside the match, so ordinary 10-to-19-digit numbers (order IDs, tracking numbers, timestamps) are left alone. Well-known system SIDs like `S-1-5-18` (SYSTEM) are preserved.

CUI markings (v1.1): see the dedicated section below.

Config File Format

The config file is a CSV with three types of rules:

Text substitution - replaces literal strings anywhere in the data:

```
# Simple replacement
sensitive-hostname,single,REDACTED_HOST

# Random replacement (picks one each time)
my-secret-domain.com,random,"example1.com,example2.com,example3.com"

# Two-column shorthand (implicit single mode)
my-company.com,example.com
```

JSON field rules (`@json` prefix) - targets specific field names at any nesting depth in JSON data:

```
@json,accessKeyId,REDACTED_KEY
@json,accountId,random,"000000000001,000000000002,000000000003"
@json,userName,REDACTED_USER
```

Key-value tag matching - handles AWS/Azure/GCP tag structures like `{"key": "Owner", "value": "admin@corp.com"}`:

```
# Use the tag's key name (not "key" itself)
@json,Owner,random,"user_a@example.com,user_b@example.com"
@json,Environment,REDACTED_ENV
```

Lines starting with `#` are treated as comments.

JSON Nested Paths (v1.1)

`@json` rules may now address nested paths using dot notation. Rules without a dot continue to match any field with that name at any depth (v1.0 behavior).

```
# Unqualified - matches "accountId" anywhere in the tree (v1.0 behavior)
@json,accountId,random,"000000000001,000000000002"

# Qualified - matches "arn" ONLY when nested under userIdentity.
# Also matches "detail.userIdentity.arn" inside a CloudWatch Events envelope.
@json,userIdentity.arn,arn:aws:iam::REDACTED:user/REDACTED
```

Qualified rules match as path suffixes anchored at dotted boundaries, so `userIdentity.arn` matches `userIdentity.arn` and `detail.userIdentity.arn` but not `foo.userIdentity.arn.extra`. This is the fix for the long-standing bug where paths like `userIdentity.arn` in v1.0 configs silently did nothing.

CUI Marking Detection (CMMC / NIST SP 800-171)

CUI (Controlled Unclassified Information) is a federal data category defined by 32 CFR Part 2002 that replaced the older patchwork of FOUO/SBU/LES markings. CMMC programs are required to protect CUI per NIST SP

800-171, and logs that *contain or mention* CUI content are themselves CUI-sensitive.

Unlike PII, CUI is identified by markings, not content patterns. When the scrubber sees a CUI banner, portion marking, or legacy marking on a value or line, it replaces the entire payload with a metadata-only placeholder:

```
Input:   CUI//SP-PRVCY Employee record: John Smith DOB 01/15/1980
Output:  [CUI-REDACTED: CUI//SP-PRVCY, 57 bytes]
```

The placeholder preserves category and byte count so downstream consumers (LLMs, dashboards, normalization tools) still see the shape of the data - without any of the content.

What's detected

Banner markings (authoritative, appear at top of documents or pasted into tickets/emails):

- `CUI`, `CUI//BASIC`
- `CUI//SP-PRVCY` (Privacy), `CUI//SP-PROPIN` (Proprietary), `CUI//SP-EXPT` (Export Controlled), `CUI//SP-LEI` (Law Enforcement), `CUI//SP-CTI` (Controlled Technical Information)
- `CONTROLLED//...` (verbose form)

Portion markings (inline, precede a paragraph or field):

- `(CUI)`, `(CUI//SP-PRVCY)`, `(U//FOUO)`, `(U//SBU)`, `(U//LES)`, `(C)`, `(U)`

Legacy markings (pre-2010 but still widespread in old docs/tickets, treated as CUI-equivalent):

- `FOUO`, `SBU`, `LES`, `OUO`, `LIMDIS`, `NOFORN`, `FEDCON`, `ORCON`

CUI-adjacent content (conservative - filenames and document titles that signal CUI-regulated content):

- `ITAR`, `EAR99`, `ECCN <code>`, `DD 254`, `FCI`

Controlling CUI scrubbing

CUI detection is **enabled by default**. To disable it (e.g., for environments where this aggressive redaction is unwanted):

```
python log_scrubber.py events.csv --no-cui
```

Or in library code:

```
from log_scrubber import Scrubber
scrubber = Scrubber(config_path='log_scrubbing_config.csv', enable_cui=False)
```

Important limitations

CUI scrubbing is a *detection-then-redact* layer. If a log line contains CUI content but no marking, the scrubber has no way to know. For defense-in-depth, combine this layer with:

1. The other built-in pattern layers (PII, credentials) for content that has pattern-matchable shape.
2. Organization-specific `@json` and text rules in `log_scrubbing_config.csv` for known sensitive fields.
3. Manual review of output before any transmission. **Always spot-check samples before sending them to an internal LLM or other downstream system.**

Library Use

`log_scrubber.py` is import-safe - the CLI banner and argument parsing are guarded by `if __name__ == "__main__"`, so `import log_scrubber` from another module has no side effects.

Class API (recommended for new code):

```
from log_scrubber import Scrubber

scrubber = Scrubber(config_path='log_scrubbing_config.csv')
safe_text = scrubber.scrub(raw_log_line)

# Batch convenience
safe_batch = scrubber.scrub_many([line1, line2, line3])

# Programmatic construction with inline rules
scrubber = Scrubber(
    text_rules=[('acme.com', 'single', 'example.com')],
    json_field_rules=[('accountId', 'single', '000000000000'),
                      ('userIdentity.arn', 'single',
'arn:aws:iam::REDACTED:user/REDACTED')],
    enable_builtins=True,
    enable_cui=True,
)
```

A single `Scrubber` instance is safe to share across threads as long as the rule lists aren't mutated after construction.

Functional API (backward compatible):

```
from log_scrubber import parse_scrubbing_config, scrub_text

text_rules, json_rules = parse_scrubbing_config('log_scrubbing_config.csv')
clean = scrub_text(raw, text_rules, json_rules)

# Optional flags (default True) for v1.1 feature control
clean = scrub_text(raw, text_rules, json_rules,
                   enable_builtins=True, enable_cui=True)
```

Existing callers that don't pass the new kwargs get the v1 behavior plus the new scrubbing layers automatically.

Example Config File

```
# === Text Substitution Rules ===
ns2.com,single,example.com
sapns2,single,examplecorp
my-splunk-server,single,splunk-host

# === JSON Field Rules ===
@json,accessKeyId,REDACTED_KEY
@json,accountId,random,"000000000001,000000000002,000000000003"
@json,userName,REDACTED_USER
@json,sourceIPAddress,10.0.0.x
@json,Owner,random,"user_a@example.com,user_b@example.com"
@json,Name,REDACTED_NAME
@json,Environment,random,"dev,staging,prod"
```

Command Reference

```
usage: log_scrubber.py [-h] [--config CONFIG] [--output OUTPUT]
                       [--no-builtin] [--no-cui] [--include-raw]
                       [--dry-run] [--quiet] [--version]
                       [mode] input [input ...]
```

Argument	Description
`mode`	Optional. Either `fieldsummary` or `samples`. Omit for auto-detection.
`input`	One or more input file paths. Glob patterns (`*.csv`) are expanded.
`--config`, `-c`	Path to scrubbing config CSV (auto-detected if not specified)
`--output`, `-o`	Output file path (single input only; ignored in batch mode). Default: `<input>_scrubbed_<timestamp>.<ext>`
`--no-builtin`	Disable built-in regex patterns (IPs, emails, tokens, SSN, CC, etc.)
`--no-cui`	Disable CUI marking detection (enabled by default)
`--include-raw`	Include the original raw values column in fieldsummary output (default: scrubbed only)
`--dry-run`	Show what would be done without writing output
`--quiet`, `-q`	Suppress informational output
`--version`	Print version and exit

Supported Input Formats

Fieldsummary Mode

Accepts two CSV formats (auto-detected):

1. **Splunk fieldsummary export** - columns: `field`, `count`, `distinct_count`, `values`, etc.
2. **Pre-scrubbed export** - columns: `Field Name`, `Raw Values`, `Scrubbed Values`, `Count`, `Distinct Count`. This is the same format the scrubber itself writes with `--include-raw`, so re-running the tool on its own output is supported.

Both formats are handled automatically. Output column headers are normalized to `Field Name` and `Scrubbed Values` regardless of input format.

Samples Mode

Handles three formats (auto-detected):

1. **CSV with `_raw` column** - standard Splunk CSV export. The `_raw` column is scrubbed in place.
2. **Plain text** - one event per line (syslog, key=value, etc.)
3. **JSON / JSONL** - single-line or multi-line JSON events. Brace depth tracking handles multi-line JSON objects that span multiple lines.

Tips

Reuse configs across projects. The config format is plain CSV - if you already have a `log_scrubbing_config.csv` for another project or tool, copy it alongside `log_scrubber.py` and it will just work.

Review output before sending. Automated scrubbing handles known patterns, but always spot-check the output for any environment-specific data the rules might have missed.

Large JSON events (GuardDuty, CloudTrail, etc.) are handled natively - the scrubber parses JSON at any nesting depth and applies `@json` rules recursively, including AWS/Azure/GCP tag structures.

Add rules incrementally. Start with the built-in regex patterns, review the output, then add `@json` and text rules to the config for anything that slipped through.

License

Paydirt and `log_scrubber.py` are licensed under the Apache License, Version 2.0. See the `LICENSE` and `NOTICE` files in this repository for the full terms.

In short: you may use, modify, and redistribute these tools, including for commercial purposes. If you fork the project, please follow the attribution requirements in the `NOTICE` file and choose a name for your fork that doesn't include "Paydirt" or "Machine Data Insights" - those names are trademarks of Machine Data Insights Inc.

Contributing

This repository is published as-is. We welcome bug reports and feature suggestions via email at the address listed on machinedatainsights.com, but **we do not accept pull requests** on the public repository. This is a deliberate policy to keep the project lightweight to maintain.

If you'd like to share improvements, please feel free to fork the project and publish your version under a different name, and let us know - we may incorporate similar improvements into the canonical version when appropriate.

Version History

v1.2.0

- April 24, 2026
- **First public release** under the Apache License 2.0.
- **Paydirt browser tool** (`Paydirt.html`): self-contained single-file scrubber with per-field highlighting in the comparison view, summary tab with category breakdowns, and downloadable template config.
- **Parity verified** between the browser tool and the CLI by `shared-tests/run_parity.py`.
- **Demo fixture** (`paydirt_demo.log` plus companion config) covering all 25 scrubbing categories, with negative test cases.
- **Reorganized repository layout** with downloadable artifacts at the project root.

v1.1.0

- April 20, 2026
- **CUI marking detection** (CMMC / NIST SP 800-171): banner, portion, legacy, and adjacent markings trigger full-value redaction with metadata-preserving placeholders.
- **Credential and token patterns**: AWS keys, GitHub PATs, Slack tokens, Stripe keys, JWTs, Google API keys, PEM blocks, HTTP Authorization headers, URL query-string credentials.
- **PII patterns**: SSN (valid ranges only), Luhn-validated credit cards and NPIs, formatted phone numbers, Windows user SIDs (well-known SIDs preserved).
- **Nested `@json` path support**: rules like `userIdentity.arn` match correctly; unqualified rules retain v1 behavior.
- **Auto-mode detection** and **batch processing** (multiple inputs, shell glob patterns).
- **Library API**: `Scrubber` class for importable use. Functional API remains backward compatible.
- **New flags**: `--no-cui`, `--version`.

v1.0.0

- March 13, 2026
- Initial release

Machine Data Insights Inc. *There's Gold In That Data!®* | machinedatainsights.com

Version 1.2.0 - April 24, 2026